

Молдавский Государственный Университет
Факультет Математики и Информатики
Департамент Информатики

Отчет по 4-й лабораторной работе по дисциплине „Java
Script”

Выполнил: студент группы IA2504

Zebeli Vasili

Chisinau 2026

Цель работы

Ознакомиться с классами и объектами в JavaScript, научиться создавать классы, использовать конструкторы и методы, а также реализовывать наследование.

Ход работы

Создание класса Item

Был создан класс Item, представляющий предмет инвентаря.

Поля класса:

- name — название предмета;
- weight — вес предмета;
- rarity — редкость предмета.

Методы класса:

- getInfo() — вывод информации о предмете;
- setWeight() — изменение веса предмета.

Было проведено тестирование объекта sword, проверена работа методов и изменение свойств объекта.

```
JS script.js > ...
1  class Item {
2      constructor(name, weight, rarity) {
3          this.name = name;
4          this.weight = weight;
5          this.rarity = rarity;
6      }
7
8      getInfo() {
9          return `Название: ${this.name}, Вес: ${this.weight}, Редкость: ${this.rarity}`;
10     }
11
12     setWeight(newWeight) {
13         this.weight = newWeight;
14     }
15 }
16
17 const sword = new Item("Steel Sword", 3.5, "rare");
18
19 console.log(sword.getInfo());
20
21 sword.setWeight(4.0);
22
23 console.log(sword.getInfo());
```

Результат:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Node.js v24.15.0
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
Название: Steel Sword, Бес: 3.5, Редкость: rare
Название: Steel Sword, Бес: 4, Редкость: rare
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> █
```

Создание класса Weapon

Был создан класс `Weapon`, наследующий свойства и методы класса `Item` с помощью `extends`.

Дополнительные поля:

- `damage` — урон;
- `durability` — прочность.

Методы:

- `use()` — уменьшение прочности оружия;
- `repair()` — восстановление прочности до 100;
- `getInfo()` — вывод полной информации об оружии.

```

23 console.log(sword.getInfo());
24
25 class Weapon extends Item {
26     constructor(name, weight, rarity, damage, durability) {
27         super(name, weight, rarity);
28
29         this.damage = damage;
30         this.durability = durability;
31     }
32 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Название: Steel Sword, Вес: 4, Редкость: rare
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
Название: Steel Sword, Вес: 3.5, Редкость: rare
Название: Steel Sword, Вес: 4, Редкость: rare
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4>

```

```

25 class Weapon extends Item {
26     constructor(name, weight, rarity, damage, durability) {
27         super(name, weight, rarity);
28
29         this.damage = damage;
30         this.durability = durability;
31     }
32
33     use() {
34         if (this.durability > 0) {
35             this.durability -= 10;
36         }
37     }
38
39     repair() {
40         this.durability = 100;
41     }
42 }

```

```

PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
Название: Steel Sword, Вес: 3.5, Редкость: rare
Название: Steel Sword, Вес: 4, Редкость: rare
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4>

```

```

39     repair() {
40         this.durability = 100;
41     }
42
43     getInfo() {
44         return `${super.getInfo()}, Урон: ${this.damage}, Прочность: ${this.durability}`;
45     }
46 }
47
48 const bow = new Weapon(
49     "Longbow",
50     2.0,
51     "uncommon",
52     15,
53     100
54 );
55
56 console.log(bow.getInfo());|

```

```

PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
Название: Steel Sword, Вес: 3.5, Редкость: rare
Название: Steel Sword, Вес: 4, Редкость: rare
Название: Longbow, Вес: 2, Редкость: uncommon, Урон: 15, Прочность: 100
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4>

```

```

57
58 bow.use();
59
60 console.log(bow.durability);

```

```

PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
Название: Steel Sword, Вес: 3.5, Редкость: rare
Название: Steel Sword, Вес: 4, Редкость: rare
Название: Longbow, Вес: 2, Редкость: uncommon, Урон: 15, Прочность: 100
90
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4>

```

```

61
62 bow.repair();
63
64 console.log(bow.durability);

```

```

Название: Steel Sword, Вес: 3.5, Редкость: rare
Название: Steel Sword, Вес: 4, Редкость: rare
Название: Longbow, Вес: 2, Редкость: uncommon, Урон: 15, Прочность: 100
90
100
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4>

```

Для проверки работы был создан объект bow, протестированы методы использования и ремонта.

Optional chaining (?.)

Была использована необязательная цепочка `?.` для безопасного обращения к свойствам объекта.

Проверено получение существующего свойства и обработка отсутствующего свойства без возникновения ошибки.

```
66  const inventory = {
67    item: {
68      owner: {
69        name: "Player1"
70      }
71    }
72  };
73
74  console.log(inventory?.item?.owner?.name);
75
76  console.log(inventory?.item?.stats?.damage);
```

100

Player1

undefined

PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> |

Функции-конструкторы

Классы `Item` и `Weapon` были переписаны с использованием функций-конструкторов.

Были использованы:

- `prototype` для добавления методов;
- `call()` для наследования свойств;
- `Object.create()` для организации наследования..

```
78 function ItemConstructor(name, weight, rarity) {
79     this.name = name;
80     this.weight = weight;
81     this.rarity = rarity;
82 }
83
84 ItemConstructor.prototype.getInfo = function() {
85     return `Название: ${this.name}, Вес: ${this.weight}, Редкость: ${this.rarity}`;
86 };
87
88 ItemConstructor.prototype.setWeight = function(newWeight) {
89     this.weight = newWeight;
90 };
91
92 const item2 = new ItemConstructor(
93     "Magic Stone",
94     1.5,
95     "rare"
96 );
97
98 console.log(item2.getInfo());
99
```

Player1

undefined

Название: Magic Stone, Вес: 1.5, Редкость: rare

PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> █

```

100     function WeaponConstructor(
101         name,
102         weight,
103         rarity,
104         damage,
105         durability
106     ) {
107         ItemConstructor.call(
108             this,
109             name,
110             weight,
111             rarity
112         );
113
114         this.damage = damage;
115         this.durability = durability;
116     }
117
118     WeaponConstructor.prototype =
119     Object.create(ItemConstructor.prototype);
120
121     WeaponConstructor.prototype.constructor =
122     WeaponConstructor;
123
124     WeaponConstructor.prototype.use =
125     function() {
126         if (this.durability > 0) {
127             this.durability -= 10;
128         }
129     };
130

```

```

136     WeaponConstructor.prototype.getInfo =
137     function() {
138         return `${ItemConstructor.prototype.getInfo.call(this)}, Урон: ${this.damage}, Прочность: ${this.durability}`;
139     };
140
141     const axe = new WeaponConstructor(
142         "Battle Axe",
143         5,
144         "legendary",
145         50,
146         90
147     );
148
149     console.log(axe.getInfo());
150
151     axe.use();
152
153     console.log(axe.durability);
154
155     axe.repair();
156
157     console.log(axe.durability);

```

```
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> node script.js
```

```
Название: Steel Sword, Вес: 3.5, Редкость: rare
```

```
Название: Steel Sword, Вес: 4, Редкость: rare
```

```
Название: Longbow, Вес: 2, Редкость: uncommon, Урон: 15, Прочность: 100
```

```
90
```

```
100
```

```
Player1
```

```
undefined
```

```
Название: Magic Stone, Вес: 1.5, Редкость: rare
```

```
Название: Battle Axe, Вес: 5, Редкость: legendary, Урон: 50, Прочность: 90
```

```
80
```

```
100
```

```
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js\лаб4> █
```

Проведено тестирование объекта axe, проверена работа методов use(), repair() и getInfo()

Контрольные вопросы

1. Насколько важен `this` в методах класса?

`this` очень важен в методах класса, потому что он указывает на текущий объект. С помощью `this` можно обращаться к свойствам и методам объекта. Например, `this.name` получает название объекта, а `this.weight` — его вес. Без `this` методы не смогут работать с данными конкретного объекта.

2. Как работает модификатор доступа `#` в JavaScript?

Символ `#` используется для создания приватных полей и методов класса. Такие свойства доступны только внутри класса и не могут использоваться снаружи объекта. Это помогает защитить данные от случайного изменения.

3. В чем разница между классами и функциями-конструкторами?

Классы — это современный способ создания объектов в JavaScript. Они более удобны и понятны, поддерживают наследование через `extends` и делают код более читаемым.

Вывод

В ходе лабораторной работы были изучены классы, объекты, наследование, методы, функции-конструкторы и необязательная цепочка в JavaScript. Были получены практические навыки создания и управления объектами, а также использования наследования и `prototype`.